CS 330, Fall 2026, Homework 7
Due Wednesday, October 21, 2026

For full credit, you must follow the guidelines laid out on the first page of homework 1.

1. **Verifying Distances.** In this problem you will design an algorithm to verify the output of Dijkstra's algorithm.
   You are given a weighted, directed graph G with *positive* edge weights and a source s. Your friend has run Dijkstra's algorithm on G from s and obtained the hash table *dist* as output. He claims that for each node v, $dist[v]$ contains the distance from s to v. He did not give you any other data—for example, the shortest paths tree (parents hash table) is not available to you.
   For some application, you need to find the shortest paths from s to each node. Unfortunately, your friend tends to write sloppy code that often returns incorrect results. For this reason, you want to double check whether the values in *dist* are correct.
   (a) Design a *linear-time* algorithm that takes as input the adjacency list G (as a nested hash table), source s, and hash table *dist*. It should return "correct" if all distances in *dist* are valid shortest-path distances from s. Otherwise, it should return "incorrect". (For full credit, your algorithm should run in linear time in the input size, i.e., in time $O(n+m)$. Algorithms that run Dijkstra's from scratch will not receive credit.)
   (b) Assuming that your algorithm from part (a) returns "correct", design a second algorithm that reconstructs the *parents* hash table, again in time linear in the input size.[1] Your algorithm may use G, s, and *dist* as needed.
   (c) In this problem, we required that all edge weights are positive. Draw an example showing that if edges of weight 0 were allowed, then your reconstruction algorithm could return a directed cycle instead of a tree.[2]
2. **Programming Problem.** See the program description in a separate PDF in the hw7 folder. Submit your programming assignment through the Gradescope autograder.

[1] It's possible to do the tasks in parts (a) and (b) in one algorithm. You can do so, but then (1) make it clear to the grader where to look for it, and (2) remember to prove that your algorithm is correct for both the first and the second tasks.

[2] There is an algorithm for part(b) that handles zero-weight edges correctly. But the most natural approach, which we assume you will use, does not.